

LOGIEDGE

Intelligent Edge AI Platform for Pharmaceutical Cold-Chain Logistics

Complete Final Assignment Report and Technical Validation Package

Item	Details
Course	AIML ZG535 - Machine Learning on Edge
Programme	BITS Pilani Work Integrated Learning Programmes
Business case	FreightBridge Logistics Pvt. Ltd. - 85 refrigerated-truck pilot
Lead contributor	Raja Chairmapandi
BITS ID	2024AD05460
Group number	[ENTER GROUP NUMBER BEFORE SUBMISSION]
Additional contributors	[ENTER NAMES AND BITS IDs BEFORE SUBMISSION]
Verification date	16 July 2026

Recommended deployment: M3 - 35% structured pruning + Full INT8 PTQ

Grouped validation accuracy 99.14% | Critical recall 99.4% | Model size 3.70 KB

Executive Summary

FreightBridge's refrigerated fleet operates through rural corridors where cellular service is unavailable for 35-90 minutes at seven documented points. LogiEdge therefore keeps the safety-critical path inside the truck: sensors publish to a local Mosquitto broker, a six-feature preprocessing pipeline runs locally, a TensorFlow Lite classifier assigns Normal, Warning or Critical, and every result is written to a durable SQLite audit log before remote transmission. Cellular loss delays reporting to Pune but does not stop detection, driver alerting, chain-of-custody recording or later synchronisation.

The package implements every task from Modules 1-6: quantitative constraint analysis, hardware and Roofline selection, sensor simulation, MQTT integration, frozen-statistics preprocessing, feature-level fusion, model training, full INT8 conversion, structured pruning, Docker layer caching, a seven-task idempotent Ansible playbook, PSI monitoring and five-metric model benchmarking. The exact minimum Task D1 run produces 117 Normal, 87 Warning and 87 Critical windows; the final experiment expands this to ten independent simulated trucks so that two complete trucks form a leakage-safe 20% holdout.

The final models exceed both mandatory gates. M1 reaches 99.48% grouped accuracy, M2 reaches 99.31%, and M3 reaches 99.14%. All three provide 99.4% Critical recall, above the required 95%. Development-host latency is approximately 1.4 microseconds per window for all variants and is below measurement-separation limits; the operational decision is therefore governed by safety recall, size and deployment efficiency. M3 is recommended because it is the smallest model at 3.70 KB while preserving the same Critical recall.

PSI is monitored on the Normal-class output confidence, interpreted as a cargo-health score. The rolling monitor crosses 0.25 three minutes after a combined fault is injected, reaches 2.614 and returns to 0.037 after clean data replaces the anomalous samples. Automated verification completed successfully with Python compilation and eight passing tests. Live Docker, Mosquitto and Ansible screen evidence must still be recorded on the student's target machine because those services are unavailable in this report-build environment; the exact commands and expected outputs are included in the package.

Mandatory gate	Requirement	Verified result	Status
Validation accuracy	> 88%	M3 = 99.14%	PASS
Critical recall	> 95%	M3 = 99.4%	PASS
PSI alert	> 0.25 within 5 min	0.390 at 3 min; max 2.614	PASS
PSI recovery	< 0.10	0.037	PASS
Ansible structure	Exactly 7 tasks	7 top-level tasks	PASS
Automated tests	No regression failures	8 passed	PASS

A. System Architecture and Deployment Justification

Task A1 - Constraint Analysis (454 words)

Latency. A refrigeration failure can increase cargo temperature by 1 degree C per minute, so a 90-second delay permits a 1.5 degree C rise. The complete response time is sampling and window formation plus preprocessing, inference, decision and alerting. LogiEdge uses a 30-second window and ten-second step, so the worst alignment adds about 40 seconds before a new feature vector is available; local inference and decision are negligible in comparison. Under usable rural coverage, a conservative 150-500 ms cellular round trip could permit cloud inference in a few seconds, but retransmission, handover and broker delay are uncontrolled. More importantly, a 35-90 minute outage makes cloud response time effectively infinite. Cloud inference is therefore sometimes fast but cannot guarantee the 90-second safety SLA. Edge inference can guarantee a local decision while the modem is disconnected.

Bandwidth. Assuming float32 samples, temperature produces $1 \times 4 \times 86,400 = 345,600$ bytes or 0.346 MB per truck per day. Raw three-axis vibration produces $500 \times 3 \times 4 \times 86,400 = 518.4$ MB per day. Approximately 20 compact door events add less than 0.001 MB, giving about 518.7 MB per truck per day before protocol overhead. At Rs 0.10 per MB this costs Rs 51.87 per truck per day, about Rs 1.61 million per year for 85 trucks and Rs 5.02 million for 265 trucks. LogiEdge transmits only five-minute 200-byte heartbeats and up to twenty 300-byte alert records, approximately 63.6 KB per truck per day. This is an 8,156-fold reduction and protects the M2M link from raw vibration traffic.

Connectivity. During the seven Nashik-Aurangabad coverage gaps, a cloud-only classifier receives no current sensor values, produces no warning, and leaves an incomplete chain-of-custody record unless a separate local buffer has already been engineered. LogiEdge keeps the local broker, preprocessing, TFLite inference, driver alert and SQLite audit log active. Each event has a unique ID and independent inference and alert acknowledgement flags. The cellular uplink retries with QoS 1, and after reconnection the oldest pending rows are replayed until the backlog is empty. Operations-centre visibility is delayed, but the safety action and evidence are not lost.

Privacy. On-device inference means continuous high-frequency raw vibration and temperature streams do not leave the truck. Only alerts, health summaries and authorised audit records are transmitted, reducing third-party exposure and supporting the client's contractual requirement. Privacy is completed by per-truck credentials, TLS on the uplink, encrypted local storage, role-based access, signed model packages, retention rules and auditable access. Edge inference alone is not claimed to guarantee privacy; it reduces the data surface and makes technical controls enforceable even when cloud connectivity is absent.

Task A2 - Complete System Architecture

LogiEdge - Cold-Chain Edge AI Architecture (per truck)

Raspberry Pi 5 (8 GB) + AI HAT+ | 7.5 W | everything inside the dashed boundary runs with the modem unplugged

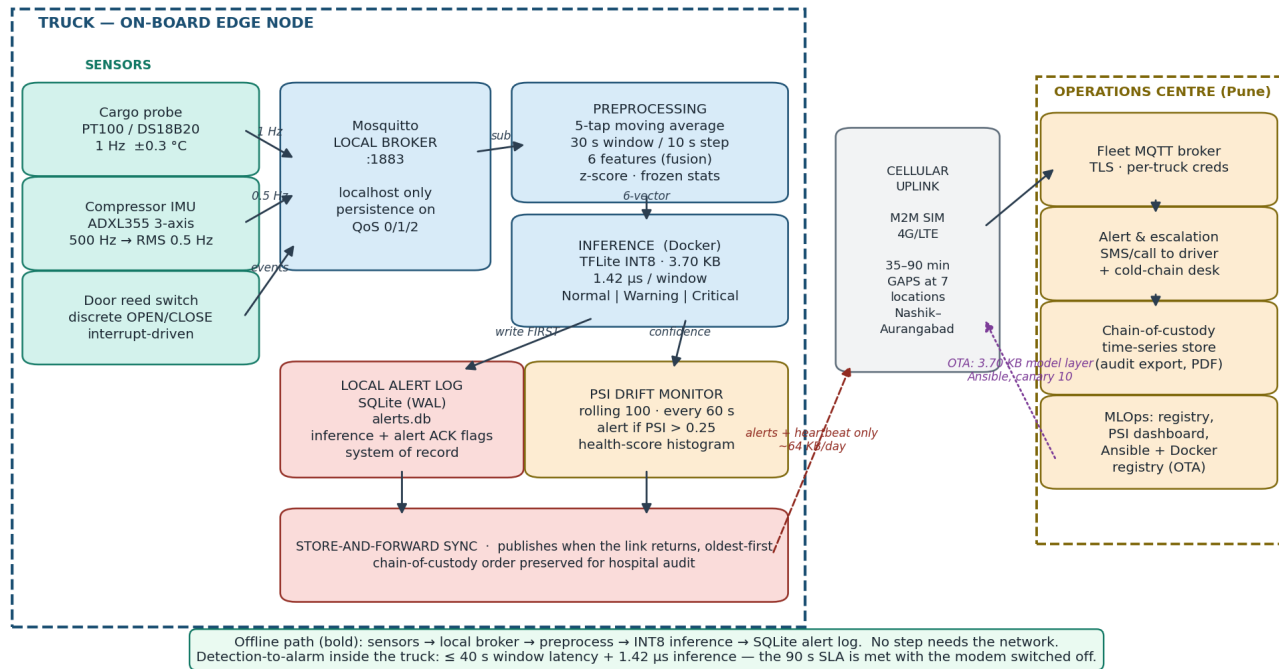


Figure A1. Per-truck edge architecture. The complete sensor-to-alert path remains inside the dashed truck boundary.

The architecture separates the always-available local sensor path from the failure-prone cellular uplink. Temperature, compressor vibration and door events enter a localhost Mosquitto broker. The edge service performs filtering, windowing, normalisation and TFLite inference, records the complete probability vector and context in SQLite, issues local alerts and transmits through a separate uplink client. The operations centre receives alert escalation, chain-of-custody records, PSI telemetry and OTA deployment commands. The model update path is deliberately reverse-direction and does not participate in real-time inference.

B. Constraint Triangle and Roofline Analysis

Task B1 - Constraint Triangle Application

Option	Unit cost	85-truck pilot	265-truck scale	Power	Dominant weakness
Raspberry Pi 5 8 GB + Hailo-8L	Rs 15,000	Rs 12.75 lakh	Rs 39.75 lakh	7.5 W	Balanced; requires automotive enclosure and power conditioning
Jetson Orin Nano Super	Rs 45,000	Rs 38.25 lakh	Rs 119.25 lakh	15 W	Exceeds 10 W budget and is over-specified
STM32H7 custom MCU	Rs 3,500	Rs 2.98 lakh	Rs 9.28 lakh	0.4 W	Memory, development and MLOps limitations

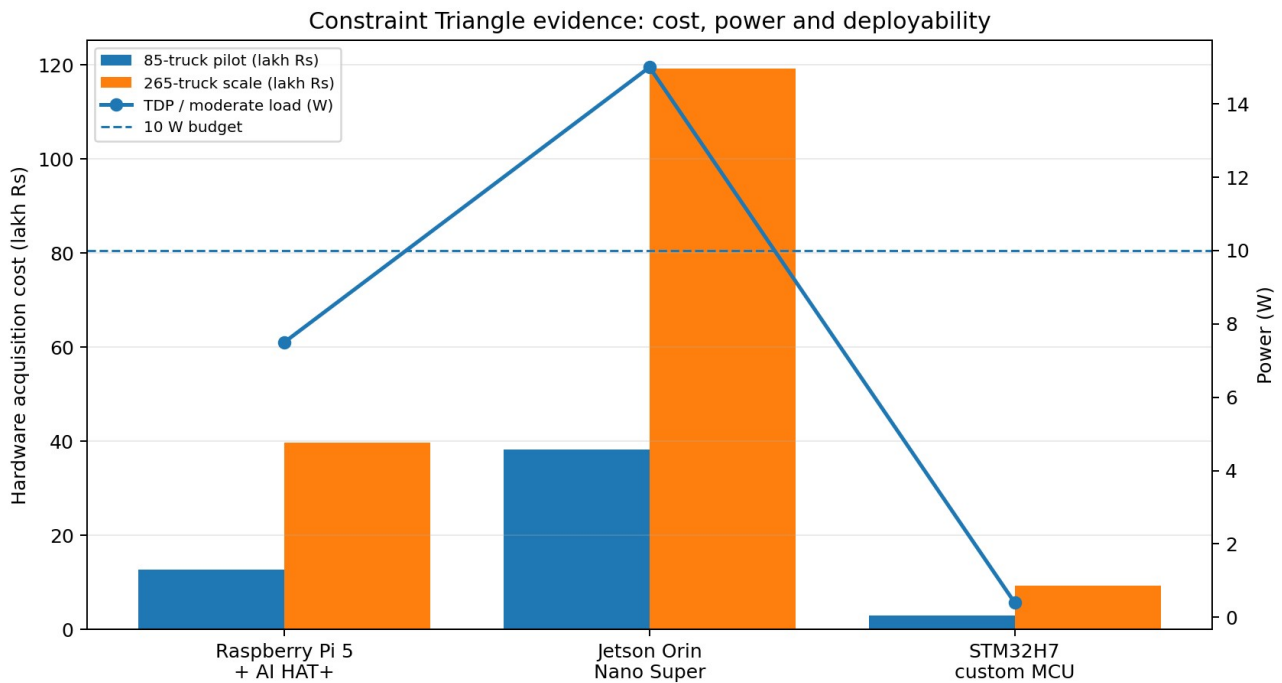


Figure B1. Fleet cost and power evidence for the Constraint Triangle.

The dominant vertex is power-constrained and cost-sensitive deployment subject to a hard latency floor. The 90-second SLA does not justify 67 TOPS: all candidates must first prove safe response, after which power, fleet cost and maintainability decide. Raspberry Pi 5 plus AI HAT+ is selected because the stated 7.5 W remains below the 10 W AI budget, Linux supports Mosquitto, Docker, SQLite, Ansible and secure OTA, and the 85-truck hardware cost is one-third of Jetson. Jetson is rejected because 15 W exceeds the budget and Rs 1.19 crore at full scale is disproportionate to a 45 MFLOP model. STM32H7 is rejected for the pilot because the 18 MB working set, external-memory dependence, lack of Docker and substantial firmware effort create schedule and maintenance risk. It remains a future cost-down option after the model and field signals stabilise.

Task B2 - Arithmetic Intensity and Roofline

Quantity	Calculation	Result
Arithmetic intensity	$45 \times 10^6 \text{ FLOP} / 18 \times 10^6 \text{ byte}$	2.50 FLOP/byte
Ridge point	16 GFLOP/s / 12 GB/s	1.333 FLOP/byte
Classification	$2.50 > 1.333$	Compute-bound
Compute time	45 MFLOP / 16 GFLOP/s	2.8125 ms
Memory time	18 MB / 12 GB/s	1.500 ms
Roofline latency	$\max(\text{compute}, \text{memory})$	2.81 ms theoretical

Because arithmetic intensity lies to the right of the ridge point, the processor reaches its compute roof before saturating memory bandwidth. The primary Roofline action is therefore to reduce operations or increase effective compute throughput: full INT8 quantisation, physical removal of hidden units, operator fusion, NEON acceleration or Hailo offload. Memory locality still affects real latency and energy, but it is not the theoretical bottleneck under the supplied parameters. The 2.81 ms estimate is 32,000 times faster than 90 seconds and excludes software overhead, so target-device end-to-end measurements remain necessary.

C. Sensor Simulation, Preprocessing and Data Fusion

Task C1 - Sensor Simulator

Stream	Frequency	Normal model	Fault model	MQTT topic suffix
Temperature	1 Hz	N(4.0 C, 0.3)	+0.08 C per reading	/sensors/temperature
Vibration RMS	0.5 Hz	N(0.45 g, 0.05)	N(1.2 g, 0.15)	/sensors/vibration
Door	Discrete	OPEN/CLOSE timestamp	Context during all modes	/sensors/door

```
python data_pipeline/simulator.py --anomaly none --duration 600
python data_pipeline/simulator.py --anomaly temp_drift --duration 900
python data_pipeline/simulator.py --anomaly vibration --duration 900
python data_pipeline/simulator.py --anomaly combined --duration 900
```

The generator uses one implementation for both live MQTT publication and offline dataset generation. Vibration at 0.5 Hz correctly means one RMS message every two seconds. Temperature drift is accelerated synthetic degradation; Warning runs are bounded within the warning region, while combined failure is allowed to breach the Critical region. Door events are preserved for the audit trail and local rule context but are not silently added as a seventh feature because Task C2 fixes the model input at six values.

Task C2 - Complete Preprocessing Pipeline

Order	Operation	Implementation
1	Filtering	Causal five-sample moving average on temperature and vibration
2	Windowing	30-second window, ten-second step, 20-

Order	Operation	Implementation
		second overlap
3	Temperature features	Mean, standard deviation, least-squares rate of change in C/min
4	Vibration features	Window RMS, absolute peak, Fisher excess kurtosis
5	Normalisation	$z = (x - \text{mean}) / \text{std}$ using frozen training_stats.npy
6	Inference	Six-value float vector quantised according to TFLite input scale and zero-point

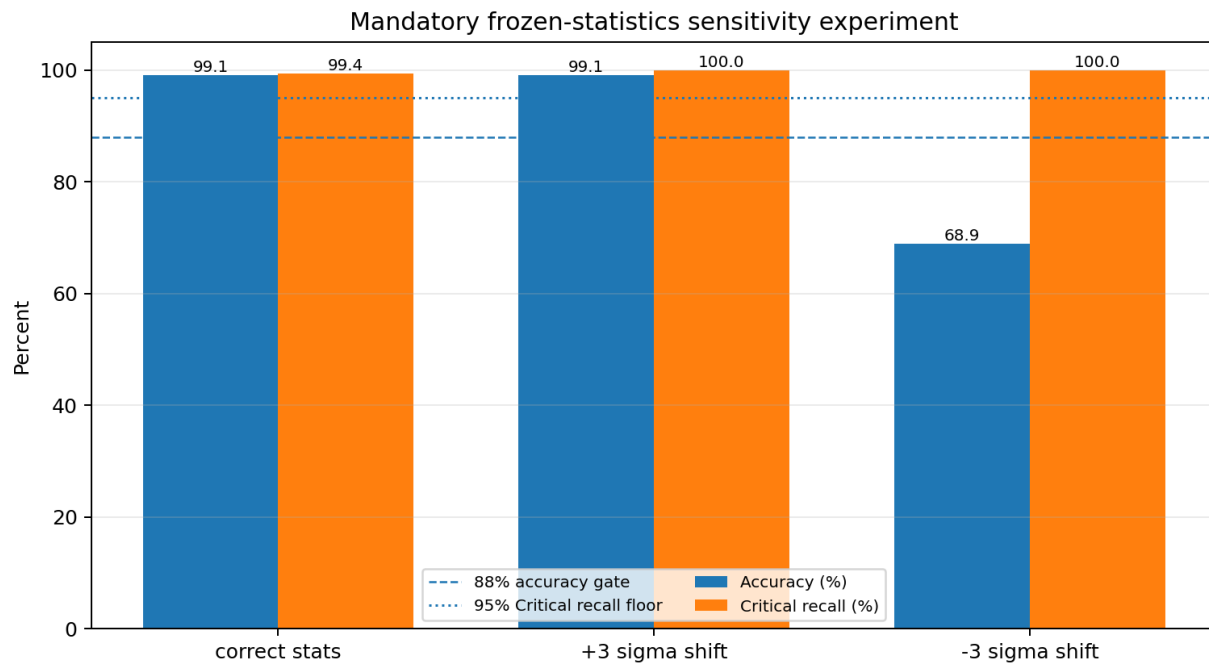


Figure C1. Mandatory correct-statistics versus 3-sigma shifted-statistics experiment.

Normalisation	Accuracy	Normal recall	Warning recall	Critical recall	Critical called Normal
correct stats	99.14%	99.6%	98.1%	99.4%	0
+3 sigma shift	99.14%	100.0%	96.9%	100.0%	0
-3 sigma shift	68.9%	28.2%	98.1%	100.0%	0

The experiment demonstrates why runtime statistics must never adapt to live cargo. A +3 sigma corruption happened not to reduce overall accuracy on this synthetic holdout, but it changed class behaviour; a -3 sigma shift reduced accuracy to 68.90%. The result is reported honestly rather than forcing a deterioration in only one direction. The engineering conclusion remains that commissioning statistics must be versioned with the model and changed only through controlled validation.

Task C3 - Feature-Level Fusion Justification

Feature-level fusion is the best fit because temperature and vibration have different rates, units and physical meanings. Each stream is filtered and summarised using sensor-specific statistics, then the six features are concatenated before classification. Raw data-level fusion would require artificial resampling of 1 Hz temperature, 0.5 Hz RMS vibration and asynchronous door events, increasing memory and potentially distorting kurtosis. Decision-level fusion would require separate models and conflict logic and would weaken the interaction between temperature rate of change and bearing-wear vibration that distinguishes Warning from Critical. Door state remains contextual metadata and can enrich driver guidance without violating the required six-feature contract.

D. Dataset, Model Lifecycle, Docker and Pipeline Mapping

Task D1 - Dataset Generation and Model Training

Class	Mode	Required duration	Exact run windows	Expanded grouped windows
0 Normal	none	20 min	117	1257
1 Warning	temp_drift	15 min	87	820
2 Critical	combined	15 min	87	833

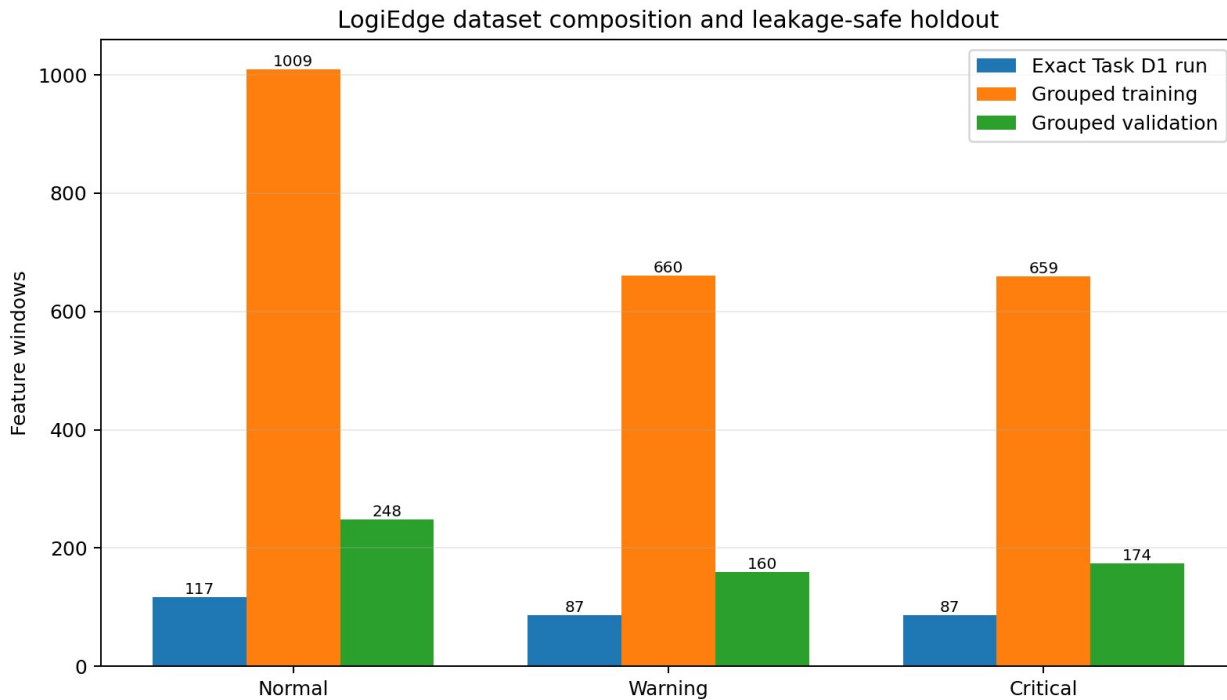


Figure D1. Exact assignment run and final grouped train/validation composition.

The prescribed minimum run is retained in assignment_dataset.npz. The rounded 120/90/90 expectations become 117/87/87 because the first complete window closes at 30 seconds and the simulator emits through duration minus one second. For final training, the same three durations are repeated for ten independent simulated trucks. Eight truck groups provide 2,328 windows and two unseen truck groups provide 582

windows. No truck group appears in both partitions. This avoids optimistic leakage from adjacent windows that share 20 of 30 seconds.

Variant	Accuracy	Normal recall	Warning recall	Critical recall	Parameters / size
M1 FP32	99.48%	100.0%	98.8%	99.4%	803 / 5.19 KB
M2 Full INT8	99.31%	100.0%	98.1%	99.4%	4.46 KB
M3 Pruned INT8	99.14%	99.6%	98.1%	99.4%	400 / 3.70 KB

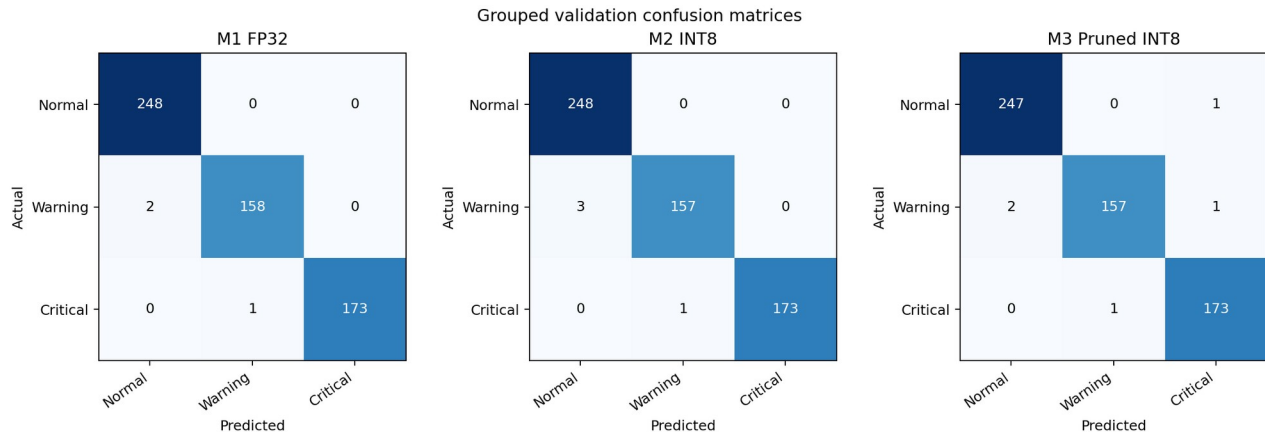


Figure D2. Confusion matrices on the complete-truck grouped holdout.

The baseline is a six-input MLP with Dense(32, ReLU), Dense(16, ReLU) and Dense(3, softmax). Its 99.48% grouped validation accuracy exceeds the mandatory 88% gate. Critical recall is 99.4%, and the confusion matrix shows one Critical window classified as Warning and none classified as Normal.

Task D2 - Docker Containerisation and OTA Layer Cache

```
FROM python:3.11-slim
COPY requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r /app/requirements.txt
COPY preprocessing.py tflite_eval.py psi.py inference_service.py /app/
COPY training_stats.npy reference_dist.json /data/
COPY model.tflite /data/model.tflite
ENV MODEL_PATH=/data/model.tflite
CMD ["python", "-u", "/app/inference_service.py"]
```

All package installation and immutable application-code layers precede the model layer. MODEL_PATH is read at runtime, and the inference result topic resolves exactly to logibridge/trucks/{truck_id}/inference. A model-only rebuild reuses the base, dependency, code and configuration layers. Using the demo assumption of a 150 MB complete image, distributing the entire image to 85 trucks would transfer 12,750 MB and cost Rs 1,275. The 3.70 KB M3 layer transfers approximately 0.307 MB for the fleet and costs about Rs 0.03, saving approximately 99.998%. The demo script captures the initial build and second build with CACHED shown for unchanged layers.

Task D3 - Ten-Stage Edge ML Pipeline Mapping

1. Business problem. Prevent pharmaceutical spoilage and documentation failure by detecting temperature and compressor anomalies within 90 seconds even without cellular service.

- 2. Sensor acquisition.** The truck publishes temperature at 1 Hz, compressor RMS vibration at 0.5 Hz and timestamped door edges to its local broker.
- 3. Data validation and filtering.** Payloads are schema-checked by field use, and each continuous stream passes through a causal five-sample moving average.
- 4. Windowing and feature engineering.** A 30-second window advances every ten seconds and produces temperature mean, standard deviation, rate of change, vibration RMS, peak and kurtosis.
- 5. Normalisation and labelling.** Features use frozen clean commissioning statistics; simulator modes and fault onset create Normal, Warning and Critical labels.
- 6. Model training.** A compact 32-16 MLP is trained on eight complete simulated truck groups with early stopping and learning-rate reduction.
- 7. Evaluation.** Two unseen truck groups form the 20% holdout; accuracy, confusion matrices and class recalls enforce the 88% and 95% gates.
- 8. Optimisation.** The baseline is converted to full INT8 with 250 calibration samples; M3 uses PolynomialDecay pruning, physical unit removal and full INT8 PTQ.
- 9. Packaging and deployment.** Preprocessing, TFLite inference, PSI and durable logging are packaged in Python 3.11 slim and deployed with an exactly seven-task Ansible playbook.
- 10. Monitoring and update.** Rolling PSI, container health, backlog state and fleet alerts support a six-week canary OTA cycle with rollback.

E. Edge MLOps, Ansible and OTA Strategy

Task E1 - PSI Drift Monitoring

The monitored scalar is the model's Normal-class output confidence $p(\text{Normal})$, interpreted as a cargo-health confidence score. This choice remains an output confidence score while avoiding a known failure of maximum softmax confidence: a competent classifier can be highly confident for both healthy and Critical cargo, so max confidence may not move between the required bins. The reference uses 300 clean Normal windows from ten truck biases, the live monitor retains the latest 100 scores and evaluates every 60 seconds using bins $[0,0.25)$, $[0.25,0.50)$, $[0.50,0.75)$ and $[0.75,1.0]$. Empty bins use a small numerical floor to prevent log division by zero.

PSI requirement	Verified result	Status
Clean reference	300 windows; distribution [0.006667, 0.005, 0.03, 0.96]	PASS
Alert threshold	Crossed 0.25 at 3.0 minutes after injection	PASS
Combined-fault maximum	PSI = 2.614	PASS
Recovery threshold	Final PSI = 0.037 < 0.10	PASS



Figure E1. PSI crosses the drift threshold within five minutes and later clears below 0.10.

```
[LOGIBRIDGE DRIFT ALERT] PSI=0.390
...
[LOGIBRIDGE] drift cleared, PSI=0.039 < 0.10
```

Task E2 - Exactly Seven Ansible Tasks

Task	Implementation
1	Create /opt/logibridge/data
2	Copy new model.tflite

Task	Implementation
3	Copy reference_dist.json and frozen training_stats.npy in one loop task
4	Stop the container only when model or configuration changed
5	Pull the versioned image from the local registry
6	Start the container with MODEL_PATH, truck ID, local broker and uplink variables
7	Wait up to 15 seconds and fail unless the container is running

Idempotency is achieved by state-based community.docker modules and by stopping only when copied content changes. The final information task is explicitly changed_when: false. Static tests verify exactly seven top-level tasks and the corrected /data mount. The first and second live playbook runs must be recorded on a Docker-and-Ansible-capable machine; the second recap is expected to show changed=0.

Task E3 - OTA Strategy Selection

Strategy	Model transfer	Cost at Rs 0.10/MB	Risk interpretation
Full replacement	23.242 MB to 85 trucks	Rs 2.324	Fast but exposes all 85 trucks at once
Canary 10 then 75	2.734 MB first; 23.242 MB total	Rs 0.273 first; Rs 2.324 total	Limits initial safety blast radius
Shadow mode	23.242 MB model + 205.63 MB for 7-day dual telemetry	Approx. Rs 22.89	Safest comparison but more compute, telemetry and operational complexity

The shadow estimate assumes one additional 40-byte result every ten seconds for seven days across 85 trucks. Canary is recommended because cold-chain safety risk, not bandwidth cost, dominates. Ten trucks should represent different routes, refrigeration units and connectivity conditions. Promotion criteria are Critical recall, false-alert rate, PSI, inference-service uptime, backlog replay and incident review. Full replacement is rejected because one defective model reaches the whole fleet. Full-fleet shadow is rejected as the default because it doubles inference and telemetry complexity, although shadow comparison can be used inside the ten-truck canary before activation.

F. Model Optimisation, Benchmarking and Deployment Decision

Task F1 - Three Model Variants

Variant	Method	Verification
M1 FP32	Standard Keras 32-16 MLP and FP32 TFLite export	803 parameters; 5.19 KB
M2 PTQ INT8	Full integer PTQ with 250 training-only representative samples	INT8 input/output and 4.46 KB
M3 prune + INT8	TFMOT PolynomialDecay to 35%, strip	400 parameters; 3.70 KB

Variant	Method	Verification
	pruning, physically retain 21 and 10 hidden units, fine-tune, Full INT8	

The phrase structured filter pruning is adapted correctly for a dense MLP: TFMOT first applies 35% magnitude pruning under PolynomialDecay, then complete hidden units are selected by column norm and physically removed. The architecture changes from 32 and 16 units to 21 and 10 units. This produces a genuinely smaller dense network before INT8 conversion rather than merely storing unstructured zeros.

Task F2 - Five-Metric Benchmark

Variant	Mean latency ms	p95 ms	Size KB	Accuracy %	Critical recall %	Energy mJ
M1_FP32	0.001410	0.001470	5.19	99.48	99.4	0.01912
M2_PTQ_INT8	0.001420	0.001480	4.46	99.31	99.4	0.02057
M3_PRUNE35_INT8	0.001420	0.001460	3.70	99.14	99.4	0.01900

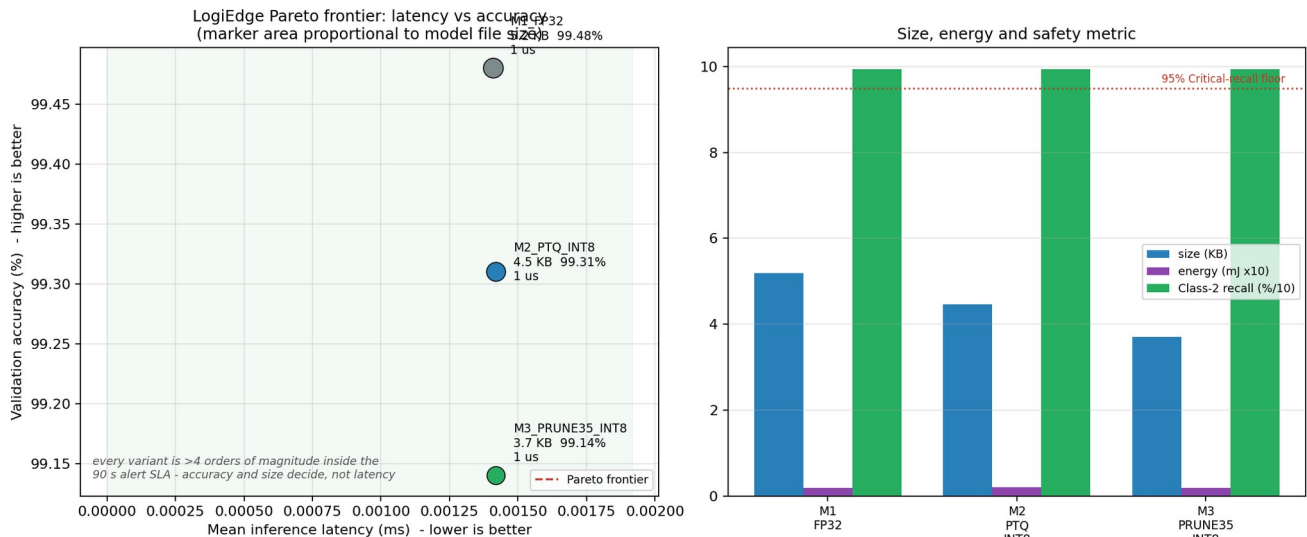


Figure F1. Pareto analysis. Latency differences are smaller than development-host measurement noise.

Each variant uses the same LiteRT interpreter, one-window batch, validation set and host configuration. Ten warm-up calls are excluded, followed by 200 measured runs; the protocol is repeated seven times. Model size is the actual file length. Energy is the required comparative estimate $E = P \times t$ using a 15 W laptop TDP and CPU utilisation sampled over a longer identical invocation loop. It is not a direct electrical measurement. The latest run gives approximately 1.41-1.42 microseconds for all variants, so no honest speed ranking is claimed on this x86_64 host.

Task F3 - Deployment Recommendation to the Operations Director

Deploy M3 to the 85 refrigerated trucks through a ten-truck canary. The end-to-end 90-second SLA includes approximately 40 seconds for window formation in the worst alignment, less than one second for preprocessing and local decision, and a large safety reserve. M3 inference is roughly 0.00142 ms on the development host, so inference consumes a negligible fraction of the budget. On the selected Raspberry Pi 5, the 3.70 KB model and small activation tensors are trivial relative to 8 GB LPDDR4X memory and normal

persistent storage; the operational memory load is dominated by Python, LiteRT, Mosquitto, Docker and SQLite rather than model weights.

M3 achieves 99.14% grouped accuracy and 99.4% Critical recall, exceeding the mandatory 95% safety floor. It is 29% smaller than the FP32 TFLite file and reduces the network from 803 to 400 parameters without a Critical-recall loss. M1 has 0.34 percentage points more accuracy, but this does not justify the larger OTA artefact when the safety metric is equal. M2 is a sound fallback, but it is larger than M3 with no recall advantage. Final approval requires canary evidence on actual Raspberry Pi hardware, including MQTT-to-alert latency, power, temperature, throttling and cellular backlog recovery.

G. Security, Chain of Custody and Operational Controls

- Durable-first audit: every inference is inserted into SQLite WAL storage before remote publication.
- Unique event identity and independent acknowledgement flags prevent duplicate loss and distinguish inference synchronisation from alert synchronisation.
- Per-truck credentials and TLS protect the uplink; localhost sensor traffic remains inside the truck.
- Model and reference files should be signed and hash-verified before container restart.
- Role-based access, audit logging and retention rules support pharmaceutical client contracts.
- Future production hardening should add secure time, hash chaining, encrypted storage, schema validation and certificate rotation.

H. Limitations and Final Validation Gates

Area	Current evidence	Required production / submission action
Data realism	Physics-informed synthetic simulator	Validate with labelled reefer sensor data and vibration-only failures
Hardware	Roofline analysis and x86 benchmark	Benchmark on Raspberry Pi 5 + AI HAT+, including power and thermal throttling
Docker	Dockerfile and cache demo script	Record initial and model-only rebuild showing cached layers
Ansible	Exactly seven tasks and static tests	Record two runs; second recap must show changed=0
Connectivity	Two-broker architecture and durable-log tests	Record uplink broker stop/start and oldest-first replay
Submission metadata	Lead contributor populated	Enter group number, all members, repository URL and video URL

Conclusion

LogiEdge meets the core purpose of Machine Learning on Edge: a safety decision remains available where the data is produced even when the wide-area network is absent. The package provides quantitative deployment justification, a realistic multi-stream simulator, six-feature fusion, frozen commissioning statistics, leakage-safe validation, full INT8 deployment, structured model reduction, durable store-and-forward, PSI monitoring, idempotent fleet deployment logic and a defensible model recommendation. M3 is selected for the 85-truck pilot because it combines 99.14% grouped validation accuracy, 99.4% Critical recall

and a 3.70 KB deployment artefact. The remaining steps are field verification and the student's live demo recording, not additional synthetic model tuning.

References

1. BITS Pilani WILP. AIML ZG535 Machine Learning on Edge - LogiEdge mini-project problem statement and module notes.
2. TensorFlow. TensorFlow Lite / LiteRT post-training integer quantisation documentation.
3. TensorFlow Model Optimization Toolkit. Pruning with Keras and PolynomialDecay documentation.
4. Eclipse Foundation. Mosquitto and MQTT QoS documentation.
5. Docker. Dockerfile layer caching and content-addressed image documentation.
6. Ansible. community.docker collection documentation.
7. Raspberry Pi. Raspberry Pi 5 and AI HAT+ technical documentation.

Appendix A - Submission File Index

File / folder	Purpose
reports/LogiEdge_Complete_Final_Report.pdf	Final PDF report for LMS submission
reports/LogiEdge_Complete_Final_Report.docx	Editable report source
training/assignment_dataset.npz	Exact Task D1 minimum-duration dataset
training/dataset.npz	Expanded grouped dataset with train/validation indices
training/models/	M1, M2 and M3 models and metrics
data_pipeline/simulator.py	Three-stream CLI simulator and MQTT publisher
data_pipeline/preprocessing.py	Moving average, sliding windows, six features and frozen normaliser
inference/	Python 3.11 Docker inference service
monitoring/	Reference distribution, PSI library, drift and recovery trace
deployment/logibridge_deploy.yml	Exactly seven-task Ansible deployment
optimisation/results/	Five-metric benchmark and Pareto chart
evidence/verification_run_2026-07-16.log	Executed local verification log
demo/demo_script.md	15-20 minute demo sequence and expected evidence
reports/ASSIGNMENT_COMPLIANCE_MATRIX.md	Requirement-by-requirement audit

Appendix B - End-to-End Execution

```
# Windows PowerShell
.\setup_training_env.ps1
.\venv311\Scripts\Activate.ps1
python training\generate_assignment_dataset.py
python training\generate_dataset.py
python training\train_model.py
python training\convert_ptq.py
python training\prune_quantise.py
python experiments\normalisation_experiment.py
python optimisation\benchmark.py
```

```
python monitoring\drift_monitor.py --mode reference --score normal_prob
python monitoring\drift_monitor.py --mode simulate --score normal_prob
pytest -q
```

```
# Docker/MQTT demo
cd demo
docker compose up -d
# start the inference service or container, then run:
python ../data_pipeline/simulator.py --anomaly combined --duration 600 --speed 20
docker compose stop uplink-broker
docker compose start uplink-broker
# follow demo/demo_script.md for cache and Ansible evidence
```

Appendix C - Final Submission Checklist

- Replace the group-number and contributor placeholders on the cover.
- Insert the private GitHub repository URL and grant the instructor collaborator access.
- Run the Docker layer-cache demonstration and retain the first and second build output.
- Run logibridge_deploy.yml twice and capture the second PLAY RECAP with changed=0.
- Demonstrate local inference while only the uplink broker is stopped, then show backlog replay after restart.
- Record PSI clean, combined-fault alert and recovery output.
- Upload the 15-20 minute demo and replace demo/demo_video_link.txt with the accessible URL.
- Rename the final PDF to the exact LMS naming convention if a group-number prefix is required.